

JavaScript Type Coercion Mastery

Type coercion is the automatic or implicit conversion of values from one data type to another (such as strings to numbers). It is one of the most misunderstood aspects of JavaScript and a favorite topic in front-end interviews at MAANG companies. Understanding coercion rules is critical for predicting output in edge cases.

1. The Three Types of Coercion

JavaScript only coerces values to three types: **String**, **Number**, and **Boolean**. There is no implicit coercion to Objects or Functions.

Target Type	Explicit Conversion	Implicit Triggers
String	<code>String(value)</code>	The <code>+</code> operator when one operand is a string. Template literals.
Number	<code>Number(value)</code>	Math operators (<code>-</code> , <code>*</code> , <code>/</code> , <code>%</code>), unary plus (<code>+val</code>), relational operators (<code>></code> , <code><=</code>).
Boolean	<code>Boolean(value)</code>	Logical operators (<code>&&</code> , <code> </code> , <code>!</code>), <code>if</code> statements, ternary operators.

2. String Coercion

Triggered by the binary `+` operator. If any operand is a string, JavaScript converts the other operand to a string and concatenates them.

```
// Explicit
String(123)    // "123"
String(null)   // "null"

// Implicit
123 + ''       // "123"
true + ' test' // "true test"
[1, 2] + ''    // "1,2" (Arrays are flattened to strings)
```

3. Number Coercion

Triggered by math operators (except binary `+` if a string is present) and loose equality comparisons. It is the most complex coercion process.

```
// Explicit
Number("123") // 123
Number("")     // 0
Number(null)   // 0
Number(undefined) // NaN

// Implicit
+'123'         // 123 (Unary plus is the fastest way to coerce to number)
12 / "6"       // 2
"5" - "2"      // 3
true * 10      // 10 (true is 1, false is 0)
```

MAANG Interview Questions: Part 1

Q1: What is the output of `[] == ![]`? Explain the execution steps.

Output: `true`

This is a classic coercion trick question. Here is how the JavaScript engine evaluates it step-by-step:

1. The logical NOT operator `!` has higher precedence. It coerces the array `[]` to a boolean. Arrays are truthy, so `![]` becomes `false`.
2. The equation is now: `[] == false`.
3. When comparing with a boolean using loose equality, the boolean is coerced to a number. `false` becomes `0`. Equation: `[] == 0`.
4. When comparing an object (array) to a primitive (number), the object is coerced to a primitive using its `toString()` method. `[]`.`toString()` results in an empty string `""`.
5. Equation: `"" == 0`.
6. When comparing a string and a number, the string is coerced to a number. `Number("")` is `0`.
7. Final comparison: `0 == 0`, which evaluates to `true`.

Q2: What is the output of `null == 0`, `null > 0`, and `null >= 0`?

```
console.log(null == 0); // false
console.log(null > 0);  // false
console.log(null >= 0); // true
```

Explanation: This exposes an inconsistency in the JavaScript specification.

- **Loose Equality (`==`):** The spec states that `null` is only loosely equal to `undefined` (and itself). It is never coerced to a number during equality checks. Therefore, `null == 0` is `false`.
- **Relational Operators (`>`, `>=`):** These operators force numeric coercion. `Number(null)` is `0`. So, `null > 0` becomes `0 > 0` (false), but `null >= 0` becomes `0 >= 0`, which is `true`.

Q3: Predict the output of the following arithmetic expressions.

```
console.log(1 + "2" + "2");
console.log(1 + +"2" + "2");
console.log("A" - "B" + "2");
```

Output & Explanation:

- `1 + "2" + "2" → "122"`. The first `+` sees a string and concatenates (`"12"`). The second `+` does the same: `"12" + "2"`.
- `1 + +"2" + "2" → "32"`. The unary `+` before `"2"` has higher precedence and coerces it to the number `2`. Then `1 + 2 = 3`. Finally, `3 + "2"` concatenates to `"32"`.
- `"A" - "B" + "2" → "NaN2"`. The `-` operator forces number coercion. `"A"` and `"B"` cannot be parsed as numbers, resulting in `NaN` (Not a Number). Then `NaN + "2"` stringifies `NaN` and concatenates it.

MAANG Interview Questions: Part 2

Q4: Explain Object to Primitive Coercion. What happens when you add `[1, 2] + [3, 4]`?

Output: `"1, 23, 4"`

When an object (like an Array) must be coerced to a primitive, JavaScript invokes two internal methods in this specific order: `valueOf()` and then `toString()`.

1. It calls `valueOf()`. By default, an array's `valueOf()` returns the array itself (which is not a primitive).
2. Since a primitive wasn't returned, it falls back to `toString()`.
3. `[1, 2].toString()` returns `"1,2"`. `[3, 4].toString()` returns `"3,4"`.
4. The `+` operator now sees two strings and concatenates them: `"1,2" + "3,4" = "1,23,4"`.

Q5: Write an object that behaves differently when used in math versus string concatenation.

Interviewers ask this to see if you can manually override the coercion mechanism using `valueOf` and `toString`.

```
const customObj = {
  value: 10,
  valueOf() {
    return this.value; // Called for math operations
  },
  toString() {
    return "The value is " + this.value; // Called for string contexts
  }
};

console.log(customObj + 5); // 15 (Invokes valueOf)
console.log(customObj * 2); // 20 (Invokes valueOf)
console.log(String(customObj)); // "The value is 10" (Invokes toString)
console.log(`${customObj}`); // "The value is 10" (Template literals use toString)
```

Q6: What is the output of `Math.max([2])` versus `Math.max([2, 3])`?

```
console.log(Math.max([2])); // 2
console.log(Math.max([2, 3])); // NaN
```

Explanation:

The `Math.max()` function requires numeric arguments. It will implicitly coerce the arguments to numbers.

- For `[2]`: Array coercion rules apply. `[2].toString()` becomes `"2"`. Then `Number("2")` becomes `2`.
- For `[2, 3]`: `[2, 3].toString()` becomes the string `"2,3"`. When JavaScript tries to convert `"2,3"` to a number using `Number("2,3")`, it fails because of the comma, resulting in `NaN`.

Pro Interview Tip: When evaluating `==`, always remember that **Primitives win**. If you compare an object to a primitive, the object will always try to become a primitive first. If you compare a string to a boolean, both will try to become numbers first. When in doubt, convert everything to numbers.

5. Infamous "Weird" Coercion Cases

Q7: What is the output of `[] + []` and [] + {}`?`

```
console.log([] + []); // "" (empty string)
console.log([] + {}); // "[object Object]"
```

Explanation:

- `[] + []`: The `+` operator forces both arrays to become primitives. `[]`.toString() is an empty string `""`. So, `"" + ""` results in `""`.
- `[] + {}`: The array becomes `""`. The object `{}` undergoes primitive coercion, calling its `toString()` method, which returns the string `"[object Object]"`. Thus, `"" + "[object Object]"` results in `"[object Object]"`.

Q8: What is the output of `{}` + []` and {}` + {}`?`

```
console.log({} + []); // 0 (in some REPLs) or "[object Object]"
console.log({} + {}); // NaN (in some REPLs) or "[object Object][object Object]"
```

Explanation: This is a notorious JavaScript quirk that depends heavily on the parsing context.

- If parsed as a statement (e.g., directly in a browser console), the first `{}`` is treated as an empty code block, not an object. The code then evaluates `+[]` (unary plus on an array), which coerces the empty array to `0`.
- If forced to be an expression (e.g., `({} + [])` or assigned to a variable), both are treated as objects. `{}`` becomes `"[object Object]"` and `[]` becomes `""`, resulting in `"[object Object]"`.
- Similarly, `{}` + {}` as a statement treats the first `{}`` as a block, leaving `+{}``. `Number("[object Object]")` is `NaN`. As an expression, it concatenates to `"[object Object][object Object]"`.

Q9: Predict the output of `true + false` and +!![]`.`

```
console.log(true + false); // 1
console.log(+!![]); // 1
```

Explanation:

- `true + false`: The `+` operator triggers numeric coercion because neither operand is a string. `true` becomes `1` and `false` becomes `0`. So, `1 + 0 = 1`.
- `+!![]`: Evaluated from right to left.
 1. `[]` is an object, meaning it evaluates as `true`.
 2. `![]` coerces it to a boolean and negates it, resulting in `false`.
 3. `!![]` negates it again, resulting in `true`.
 4. The unary `+` coerces the boolean `true` into a number: `1`.